Cascadia

--Dev

CUI Xiang
CAO Yunliang

Introduction

Le projet met en œuvre une version console du jeu Cascadia, une version carrée et une version hexagonale, avec deux modes de jeu principaux : le mode solo et le mode multijoueur. Ainsi que plusieurs méthodes de calcul des scores. Le code est modulaire et possède une structure MVC de base, avec plusieurs paquets et classes, chacun ayant des responsabilités définies. Ce document détaille l'architecture du code et ses principaux composants.

Présentation des fonctionnalités des packages

1. bearscore (Package des règles de score pour les ours)

- ***Fonctionnalité :*** Définit les règles de score liées aux ours. Chaque classe représente une méthode de calcul de score différente.
- ***Fichiers inclus :***
  - **BearA.java :** Implémente la règle de score de base pour les ours.
  - **BearB.java :** Calcule le score en fonction des combinaisons de groupes d'ours.
  - **BearC.java :** Règle de score liée à l'expansion des groupes d'ours.
  - **BearD.java :** Règle complexe de score basée sur les interactions entre groupes d'ours.

---

2. eaglescore (Package des règles de score pour les buses)

- ***Fonctionnalité :*** Définit les règles de score liées aux buses.
- ***Fichiers inclus :***
  - **EagleA.java :** Stratégie de score de base pour les buses.
  - **EagleB.java :** Règle de score basée sur les relations de voisinage des buses.
  - **EagleC.java :** Stratégie de score avancée pour les buses.
  - **EagleD.java :** Règle de score plus complexe pour les buses.

---

3. deerscore (Package des règles de score pour les wapitis)

- ***Fonctionnalité :*** Définit les règles de score liées aux wapitis.
- ***Fichiers inclus :***
  - **DeerA.java :** Règle de score de base pour les groupes de wapitis.
  - **DeerB.java :** Calcule le score en fonction de l'alignement des wapitis en ligne droite.
  - **DeerC.java :** Règle de score avancée basée sur les

combinaisons de wapitis.
    - **DeerD.java :** Règle complexe de score basée sur les interactions des groupes de wapitis.

---

4. salmonscore (Package des règles de score pour les saumons)

- ***Fonctionnalité :*** Définit les règles de score liées aux saumons (saumons).
- ***Fichiers inclus :***
    - **SalmonA.java :** Règle de score de base pour les groupes de saumons.
    - **SalmonB.java :** Calcule le score en fonction des paires de saumons.
    - **SalmonC.java :** Stratégie de score avancée pour les combinaisons de saumons.
    - **SalmonD.java :** Règle complexe de score pour les interactions des saumons.

---

5. foxscore (Package des règles de score pour les renards)

- ***Fonctionnalité :*** Définit les règles de score liées aux renards.
- ***Fichiers inclus :***
    - **FoxA.java :** Stratégie de score de base pour les renards (basée sur les types d'animaux adjacents).
    - **FoxB.java :** Règle de score basée sur les paires d'animaux adjacents.
    - **FoxC.java :** Calcule le score en fonction de la diversité maximale des animaux adjacents.
    - **FoxD.java :** Règle complexe de score pour les interactions des renards.

---

6. game (Package de la logique principale du jeu)

- ***Fonctionnalité :*** Gère la logique principale du jeu, incluant les données du jeu et les fonctionnalités de base.
- ***Fichiers inclus :***
    - **Animal.java :** Définit les attributs et comportements de base des animaux.
    - **GameInitial.java :** Initialise les données et les configurations du jeu.
    - **Main.java :** Point d'entrée du programme pour lancer le jeu.
    - **Player.java :** Gère les données et comportements des joueurs.
    - **Tile.java :** Définit les propriétés des tuiles du jeu.

---

7. gamegraphic (Package de l'interface graphique du jeu)

- ***Fonctionnalité :*** Implémente l'interface utilisateur graphique (GUI) du jeu.
- ***Fichiers inclus :***
  - **Completecontroller.java** : Contrôle de l'interface graphique pour hexagonale.
  - **Completeview.java** : Affichage complet de la vue du jeu de hexagonale.
  - **Gamecontroller.java** : Contrôle de l'interface graphique pour carrée.
  - **GameData.java** : Gestion des données pour l'interface graphique.
  - **Gameview.java** : Affichage complet de la vue du jeu de carrée..

---

8. gameterminal (Package d'interaction via le terminal)

- ***Fonctionnalité :*** Fournit un mode d'interaction avec le jeu via le terminal.
- ***Fichiers inclus :***
  - **GameControl.java** **:** Logique de contrôle du jeu en mode terminal.
  - **GameControlSolo.java** **:** Logique de contrôle pour le mode solo.
  - **GameData.java** **:** Gestion des données en mode terminal.

---

9. score (Package de calcul de score général)

- ***Fonctionnalité*** **:** Définit les interfaces et stratégies de calcul de score génériques.
- ***Fichiers inclus*** **:**
  - **FamilyScore.java** **:** Stratégie de score de base pour la version familiale.
  - **MediumScore.java** **:** Stratégie de score pour un niveau de difficulté moyen.
  - **ScoreCalculate.java** **:** Interface de calcul de score (interface abstraite pour toutes les règles de score).

Classes principales et leurs responsabilités

1. GameControl (Contrôleur de Jeu)

- ***Rôle :*** Gère l'ensemble du flux de jeu, y compris l'initialisation des ressources, la gestion des tours des joueurs, l'application des règles du jeu et le calcul final des scores.
- ***Fonctionnalités Principales :***
  - Initialisation du jeu :

        - Génération de la piscine initiale de tuiles (***initialTilePool***), de la piscine principale de tuiles (***tilePool***) et de la piscine de jetons animaux (***animalPool***).
        - Mélange aléatoire des tuiles et des jetons animaux.
        - Attribution des tuiles initiales aux joueurs.
    - Gestion des tours :
        - Propose 4 combinaisons aléatoires de tuiles et de jetons animaux.
        - Permet aux joueurs de choisir une tuile et de placer un jeton animal.
        - Valide les règles de placement (ex : les tuiles doivent être adjacentes).
    - Calcul des scores :
        - Applique différentes règles de calcul selon le mode de jeu choisi (via l'interface ***ScoreCalculate***).

---

2. Player (Joueur)

- ***Rôle :*** Représente un joueur dans le jeu et gère ses données de jeu et ses actions.
- ***Propriétés Principales :***
    - ***tiles*** : Stocke toutes les tuiles placées par le joueur.
    - ***name*** : Le nom du joueur.
    - ***Position*** : Enregistre les coordonnées des tuiles sur le plateau du joueur.
    - ***Animalset*** : Associe les jetons animaux placés sur chaque tuile.
- ***Méthodes Principales :***
    - ***placeTile(Tile tile, int x, int y)*** : Place une tuile aux coordonnées spécifiées.
    - ***canPlaceTile(int x, int y)*** : Vérifie si une tuile peut être placée à l'endroit choisi.
    - ***addAnimalToTile(Tile tile, Animal animal)*** : Place un jeton animal sur une tuile.

---

3. Tile (Tuile)

- ***Rôle :*** Représente une tuile de paysage dans le jeu, incluant son type de paysage et les animaux qui peuvent y être placés.
- ***Propriétés Principales :***
    - ***landscape*** : Type de paysage (ex : forêt, rivière).
    - ***animals*** : Animaux pouvant être placés sur la tuile.
    - ***id*** : Identifiant unique de la tuile.
    - ***color*** : Couleur de la tuile (associée au type de paysage).
- ***Méthodes Principales :***

- ***assignLandscapeColor(String landscape)*** : Attribution automatique de la couleur selon le type de paysage.

---

4. Animal (Jeton Animal)

- ***Rôle :*** Représente un jeton animal avec un nom et une couleur spécifique.
- ***Propriétés Principales :***
  - ***name*** : Le nom de l'animal (ex : Ours, Renard, Saumon ...).
  - ***color*** : La couleur associée à l'animal.
- ***Méthodes Principales :***
  - ***assignAnimalColor(String name)*** : Attribution automatique de la couleur selon le type d'animal.

---

5. Interface ScoreCalculate et Ses Implémentations

- ***Rôle :*** Fournit un mécanisme flexible de calcul des scores, adaptable selon les règles et le mode de jeu.
- ***Implémentations :***
  - ***FamilyScore*** : Implémente les règles de calcul des scores pour le mode familial.
  - ***MediumScore*** : Implémente les règles de calcul pour le mode intermédiaire.
  - ***Bear*** A/B/C/D, ***Fox*** A/B/C/D, etc. : Implémentent différentes stratégies de calcul pour les animaux dans le mode "Carte de score animal".
- ***Méthode Principale :***
  - ***calculateScore(Player player)*** : Calcule le score total d'un joueur en fonction de ses tuiles et des jetons animaux placés.

---

6. Completeview et Gameview (Vue du Jeu)

- ***Rôle :*** Responsable de l'affichage graphique du jeu et de la gestion des interactions utilisateur. Elle s'occupe du rendu de la zone du joueur, des tuiles de paysage, des jetons animaux et des informations de progression du jeu.
- ***Fonctionnalités Principales :***
  - ***renderGame(ApplicationContext context)***
    - Définition des dimensions de la fenêtre (1000×800) et des tailles des tuiles et des cellules.
    - Rendu de l'arrière-plan, de la zone du joueur, des tuiles et des jetons animaux.
  - ***drawPlayer(Graphics2D graphics)***
    - drawPlayerplace(Graphics2D graphics) : Dessin de la grille de placement des tuiles pour le joueur (grille de 8x5).

        - Affichage des tuiles déjà placées et des jetons animaux associés.

    - ***drawTiles(Graphics2D graphics, List<Tile> tilePool)***
        - Dessin des Tuiles : Affichage des tuiles disponibles dans la réserve (couleur représentant le type de paysage).
    - ***drawCurrentStep(Graphics2D graphics)***
        - Affichage des étapes en cours et des consignes pour guider le joueur dans ses actions.
    - ***renderGame(ApplicationContext context)***
        - Actualisation de l'interface en temps réel selon les actions du joueur, avec mise à jour dynamique des tuiles et des jetons animaux.
    - ***renderGame(ApplicationContext context)***
        - Utilisation de la méthode ***renderGame()*** pour rendre l'ensemble de l'interface du jeu de manière fluide et interactive.

---

7. **GameData (Données de Jeu)**
    - ***Rôle :*** Gère toutes les données du jeu, y compris les réserves de tuiles et de jetons animaux, les informations des joueurs, ainsi que l'état global du jeu.
    - ***Fonctionnalités Principales :***
        - **Gestion des Réserves de Tuiles et de Jetons Animaux :**
            - ***addTileToPool(Tile tile)*** : Ajoute une tuile à la réserve principale.
            - ***ChuffleTilePool()*** : Mélange aléatoirement les tuiles et les jetons animaux.
            - ***removeLastTileFromPool()*** : Retire la dernière tuile de la réserve principale.
        - **Gestion des Joueurs :**
            - ***createplayer()*** : Crée les joueurs et les ajoute à la liste des joueurs selon le nombre sélectionné.
            - ***changerCurrentPlayer()*** : Passe au joueur suivant après chaque tour.
        - **Gestion des Tuiles et des Animaux sur le Plateau :**
            - ***isTileplaced(Tile tile)*** : Vérifie si un jeton animal a déjà été placé sur une tuile donnée.
            - ***setPosition(int x, int y)*** : Définit la position d'une tuile sélectionnée sur le plateau.
            - ***findTileByPosition(Player player, int x, int y)*** : Recherche une tuile placée à une position spécifique.
            - ***isAnimalValidForTile(Tile tile, Animal animal)*** : Vérifie si

un animal peut être placé sur une tuile en fonction de son type de paysage.

- **Combinaisons Tuiles-Animaux :**
  - ***generateNewCombination()*** : Génère une nouvelle combinaison aléatoire d'une tuile et d'un jeton animal.
  - ***canPlaceAnimalOnBoard(Player player, Animal animal)*** : Vérifie si un joueur peut placer un animal sur une tuile valide.
- **Gestion de l'État du Jeu :**
  - ***getCurrentStep()*** : Gère les étapes actuelles du jeu, utiles pour les instructions affichées au joueur.
  - ***geterror()*** : Gère les erreurs de placement ou d'action dans le jeu.

---

8. Completecontroller et Gamecontroller(Contrôleur Principal du Jeu Graphique)

- ***Rôle :*** Responsable de la **logique principale du jeu en mode graphique** et des interactions avec les joueurs. Il contrôle l'ensemble du déroulement du jeu, de l'initialisation des ressources, en passant par la gestion des tours des joueurs, jusqu'à la fin de la partie.
- ***Fonctionnalités Principales :***
  - **Initialisation du Jeu :**
    - ***initializeGame()*** : Initialise les ressources du jeu, mélange la pioche de tuiles et met en place les tuiles de départ.
    - ***getPlayerCount()*** : Récupère et valide le nombre de joueurs (entre 2 et 4).
  - **Gestion des Tours :**
    - ***handleGame(ApplicationContext context)*** : Gère les tours des joueurs et met à jour l'affichage du jeu en temps réel jusqu'à la fin de la partie.
    - ***ChangeTurn()*** : Change le joueur actif, réinitialise les ressources pour le tour suivant et rafraîchit l'interface.
  - **Gestion des Interactions Joueur :**
    - ***checkevent(ApplicationContext context)*** : Gère les événements clavier et souris, en répondant aux actions des joueurs sur l'interface graphique.
    - ***handlePointerEvent(ApplicationContext context, PointerEvent event)*** : Traite les clics des joueurs pour la sélection et le placement des tuiles et des jetons animaux.
  - **Placement des Tuiles et des Jetons Animaux :**
    - ***placeTile(String player, Tile chosenTile, int x, int y)*** : Permet aux joueurs de placer une tuile sur le plateau et

met à jour l'affichage en temps réel.
        - ***placeAnimal(String player, Animal chosenAnimal, int x, int y)*** : Permet aux joueurs de placer un jeton animal sur une tuile valide avec un retour immédiat.
    - **Validation des Placements :**
        - ***isPlacementValidTile(String player, int x, int y)*** : Vérifie si la tuile peut être légalement placée à la position donnée.
        - **areTilesAdjacent(Point point, int x, int y)** : Vérifie si la tuile à placer est adjacente à une tuile existante.
    - **Gestion des Ressources :**
        - ***SetTilepool()*** : Met à jour la pioche de tuiles et propose 4 tuiles aux joueurs à chaque tour.
        - ***SetAnimalpool()*** : Met à jour la pioche des jetons animaux en évitant les combinaisons de 4 jetons identiques.

Corrections et améliorations des versions précédentes

1. Amélioration de la Version Terminal

- Ajout de la sélection entre le Mode Solo et le Mode Multijoueur au démarrage du jeu.
- En Mode Multijoueur, les joueurs peuvent désormais choisir librement le nombre de participants (2 à 4 joueurs), contrairement à la version précédente limitée à deux joueurs.

2. Système de Score Complètement Révisé

- Implémentation complète des méthodes de calcul de score pour les modes Famille et Intermédiaire dans la version terminale.
- Ajout de 5 cartes de score animal (Ours, Cerf, Renard, Saumon, Aigle), chacune proposant 4 variantes de scoring (A, B, C, D), offrant ainsi plus de diversité stratégique.

3. Amélioration Complète des Versions Graphiques

- La version en grille carrée prend désormais en charge le calcul automatique des scores et la correction automatique des erreurs, permettant une expérience de jeu complète et fluide.
- Introduction d'une version simplifiée en grille hexagonale, offrant aux joueurs la possibilité d'expérimenter un gameplay plus fidèle aux règles officielles.

4. Optimisation des Structures de Données et des Performances

- Refactorisation des classes clés (***Tile, Animal***) en types record pour améliorer la gestion des données.
- Ajustement des modificateurs final et static sur plusieurs champs pour renforcer la robustesse et la stabilité du programme.

5. Optimisation et Refactorisation du Code

- Réorganisation des méthodes complexes comme ***gameRound*** et ***endGame*** : découpage des longues fonctions en sous-fonctions plus courtes pour simplifier la lecture et faciliter la maintenance.

Impact Global des Améliorations

- **Plus de Flexibilité de Jeu** : Diversité des modes de jeu, des règles de score et du nombre de joueurs.
- **Expérience Complète** : La version graphique offre désormais une expérience de jeu fluide et immersive.
- **Stabilité et Performance Accrues** : Optimisation des structures de données pour une meilleure efficacité et fiabilité.
- **Code Simplifié et Modulable** : Organisation claire facilitant les évolutions et les mises à jour futures.

Améliorations futures

1. Actuellement, ce projet ne permet de compléter l'ensemble du processus de jeu que dans la version terminale et la version avec des tuiles carrées. Cependant, dans la version hexagonale, seules les fonctionnalités de base comme le placement des tuiles et des jetons animaux sont disponibles. Le calcul final des scores dans cette version n'a pas encore été entièrement implémenté. Cela constitue l'un de nos objectifs futurs.

2. Actuellement, ce projet ne permet d'attribuer qu'un seul type de paysage par tuile. La possibilité d'avoir plusieurs types de paysages sur une même tuile est un objectif que nous visons à atteindre dans le futur.

3. Pour le moment, le projet n'intègre pas encore les jetons Nature. Que ce soit dans le déroulement du jeu ou dans le calcul final des scores, cette fonctionnalité n'a pas encore été introduite. Son implémentation fait partie de nos objectifs futurs.

4. Actuellement, dans la version hexagonale de ce projet, la rotation libre des tuiles hexagonales n'a pas encore été réalisée. Cela représente également un de nos objectifs futurs.

Merci beaucoup !